

Customer Support Resolution Agent

Technical Documentation

Multi-Agent System - Design, Components & Setup Guide

Project	Customer Support Resolution Agent
Document Type	Technical Documentation / Handover Guide
Scope	Multi-agent system - Supervisor, Data, and Action agents
Team	PwC · Cloud, Engineering & Data Analytics
Author	Akarsh
Version	3.0

1. Introduction

1.1 Purpose of This Document

This document is the technical documentation and handover guide for the Customer Support Resolution Agent. It explains what the system does, how it is built, how the components work together, and how to set up and run it. It is written so that an engineer new to the project can understand the design and continue the work without prior context.

1.2 What the System Is

The Customer Support Resolution Agent is a multi-agent AI system that helps a support team resolve customer queries. A representative asks a question in plain English; the system interprets it, retrieves the relevant information from the enterprise data platform, and returns a grounded answer based on real data.

The system is built as multiple specialist agents coordinated by a Supervisor. A Supervisor agent receives the question, decides which specialists are needed, calls them, and combines their results into one answer. Two specialists are in place: a Data Agent that retrieves billing, invoice, and account information, and an Action Agent that performs real operations such as correcting an overbilled invoice, creating a support case, or updating an account status. A React frontend with a FastAPI backend provides the chat interface, which calls the Supervisor.

1.3 The Problem It Solves

Support representatives normally answer billing questions by manually checking multiple systems and piecing the information together. This is slow, inconsistent, and hard to audit. The system automates both the retrieval and the follow-up actions - it answers specific lookups and open-ended analytical questions from governed enterprise data, and it can carry out the resulting support operations, all from a single request.

2. System Architecture

The system is organised into four layers: the user interface, the agent orchestration layer (the Supervisor and its specialist agents), the MCP server layer, and the Databricks platform. A request flows down through them and the answer flows back up. The diagram below shows the structure; each layer is described underneath.

Customer Support Agent — System Architecture

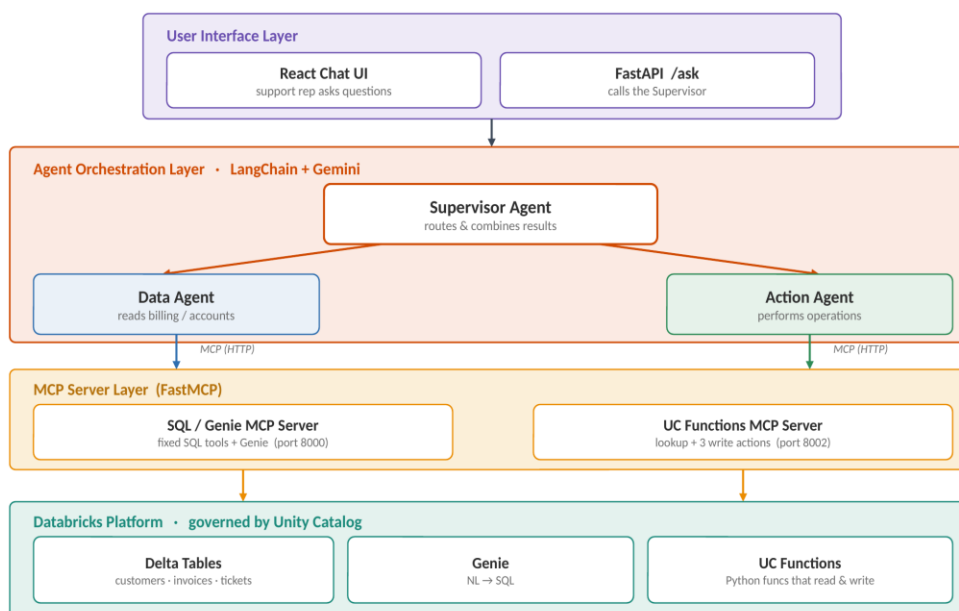


Figure 1 - Multi-agent architecture: Supervisor orchestrating Data and Action agents

2.1 User Interface Layer

Where the support representative interacts with the system. A React chat interface runs in the browser; the representative types a question and sees the grounded answer. The React app sends the question to a FastAPI backend, which calls the agent and returns the answer as JSON. This layer is what makes the agent usable as a product rather than a script.

2.2 Agent Orchestration Layer

The decision-making layer, built on LangChain with Google Gemini. At the top sits the Supervisor agent: it interprets the request, decides which specialist agents are needed, calls them, and combines their results. Beneath it are two specialist agents - the Data Agent (retrieval) and the Action Agent (operations). Each specialist is itself a LangChain agent with its own tools. The model is swappable without changing this logic.

2.3 MCP Server Layer

Two custom servers, built with FastMCP, expose the enterprise capabilities as standardised tools that the agents discover at runtime. The SQL / Genie MCP server (port 8000) serves the Data Agent: fixed SQL tools for precise lookups and a Genie tool for open-ended analytical questions. The UC Functions MCP server (port 8002) serves the Action Agent: a customer lookup plus three operations that change data. Each specialist connects to its own MCP server over HTTP.

2.4 Databricks Platform Layer

The governed data backbone. A serverless SQL warehouse serves three Delta tables (customers, invoices, support tickets). Genie provides natural-language-to-SQL over the same tables. Unity Catalog Functions - Python functions registered in Unity Catalog - perform the read and write operations the Action Agent calls. Access is governed by Unity Catalog throughout.

3. Components - What Each Part Does

This section describes each component and how they work together.

3.1 The Data Agent (Reasoning)

The Data Agent runs an iterative reason-and-act loop: read the question, reason about what is needed, choose a tool, call it, read the result, and either call another tool or compose the answer. No routing rules are hardcoded - it decides which tool fits each question by reading the tool descriptions. This decision-making is what makes it agentic rather than a fixed script. It only reads data; it never changes anything.

3.2 The MCP Server

MCP (Model Context Protocol) is the integration layer between the agent and the data. Rather than wiring the agent directly to Databricks, the MCP server exposes capabilities as standardised tools that any agent can discover and call. This keeps the agent decoupled from the data source, lets tools be added or swapped without changing the agent, and establishes the pattern that the future Search and Action agents will reuse. It is built with FastMCP and runs as an independent service the agent reaches over HTTP.

3.3 Tool Design - Two Complementary Paths

The MCP server deliberately exposes two kinds of data tools:

- Fixed, parameterised tools - pre-defined safe queries for common precise lookups (look up a customer, get invoices for a customer or month, list overbilled invoices). Fast and fully predictable; they use parameterised queries to prevent SQL injection.
- Genie tool - a natural-language-to-SQL capability for open-ended analytical questions the fixed tools cannot anticipate. The agent passes the plain-English question to Genie, which generates and runs the SQL.

The agent routes each question automatically: precise lookups go to the fixed tools, novel analytical questions go to Genie. Together they cover both the predictable and the open-ended needs of a support desk.

3.4 Genie Integration

Genie answers questions that were never explicitly programmed - ranking customers by overbilling, computing averages, and similar. A dedicated Genie Space is configured over the support data with business context so it interprets questions correctly. Because Genie is asynchronous (it takes time to generate and run SQL), the integration submits the question and then polls until the result is ready.

3.5 Grounding & Reliability

The agent is instructed to answer strictly from tool results, to state the billed amount, expected amount, difference and reason for any overbilling, and to say clearly when the data does not contain an answer. This keeps every response traceable to real data and prevents invented figures.

3.6 Frontend & API

The user-facing layer has two parts. The FastAPI backend exposes a single /ask endpoint: it receives a question, runs the agent, and returns the answer as JSON. The React frontend is a chat interface - the representative types a question, it is sent to /ask, and the answer is displayed. The frontend also offers example questions and a new-chat action. Each question is handled independently (no conversational memory at this layer); multi-turn context, if needed, would be handled by the Supervisor agent later.

3.7 The Supervisor Agent (Orchestration)

The Supervisor is the orchestrator and the heart of the multi-agent design. It receives the user's request and treats each specialist agent as a tool it can call: ask_data_agent for retrieval and ask_action_agent for operations.

It reasons about what the request needs, calls the appropriate specialist(s) - sometimes more than one, in sequence - and combines their results into a single grounded answer.

For example, the request “why was Acme overbilled, and open a case for it” leads the Supervisor to first call the Data Agent (to establish the billing facts), then call the Action Agent (to create the case using those facts), and finally merge both into one reply. A pure lookup calls only the Data Agent; a pure operation calls only the Action Agent. The routing is decided by the Supervisor's reasoning, not by hardcoded rules.

Wrapping each specialist as a tool keeps the design extensible: a further specialist (such as a Search agent) can be added as one more tool without changing the Supervisor's logic.

3.8 The Action Agent (Operations)

The Action Agent performs real, state-changing support operations - it is the part of the system that does things rather than only reading. It is a LangChain agent like the Data Agent, but its tools call Unity Catalog Functions that modify data. It exposes four tools: look up a customer; apply a correction to an overbilled invoice (updates the billed amount and marks it Corrected); create a support case (inserts a new ticket row); and update a customer's account status. Each operation changes the underlying Delta tables and is verifiable in Databricks afterwards.

3.9 The UC Functions MCP Server

The Action Agent's capabilities are exposed through a second MCP server (port 8002), separate from the SQL / Genie server. Its tools call Unity Catalog Functions registered in Databricks. The read function (customer lookup) is a SQL function; the three write operations are Python Unity Catalog Functions, because SQL functions in Unity Catalog cannot modify tables. The Python functions authenticate explicitly and run the corresponding UPDATE or INSERT against the Delta tables. This keeps the Action Agent's operations governed and registered as Unity Catalog objects rather than ad-hoc queries.

4. Request Flow

The sequence diagram below shows the messages exchanged for a combined question that needs both specialists - the fullest path through the system. Simpler questions follow a shorter version of the same flow. The walkthrough follows underneath.

Customer Support Agent — Request Sequence (combined question)

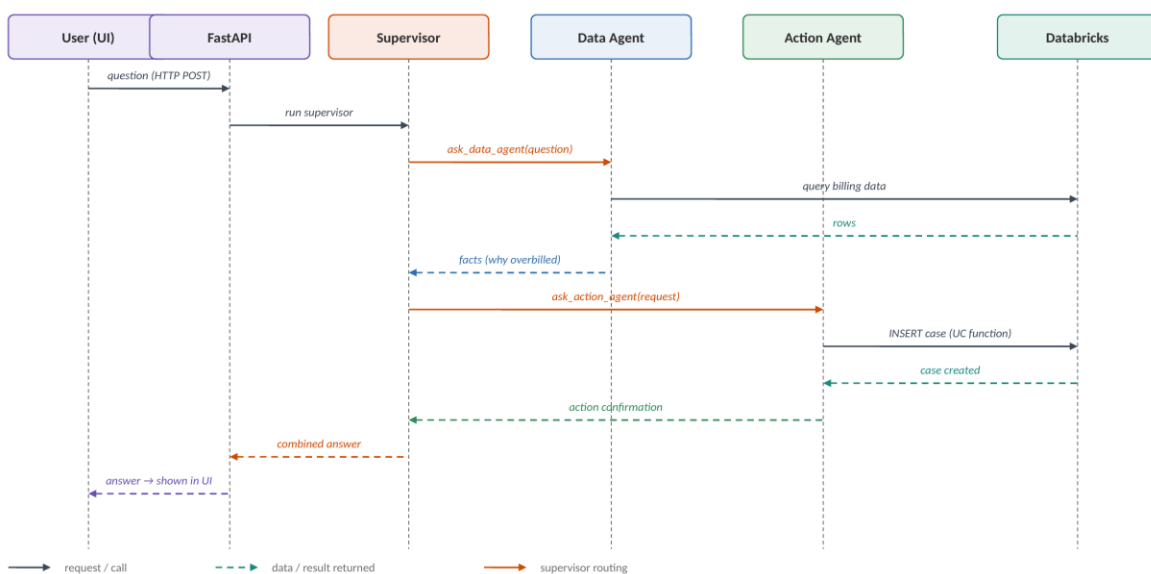


Figure 2 - Request sequence for a combined question (Supervisor routing to both specialists)

1. The user types a question in the React UI, which sends it to the FastAPI /ask endpoint.
2. FastAPI calls the Supervisor with the question.
3. The Supervisor reasons about what the request needs and calls the Data Agent first (ask_data_agent) to gather the facts.
4. The Data Agent queries Databricks through its MCP server and returns the billing facts to the Supervisor.
5. The Supervisor then calls the Action Agent (ask_action_agent) to perform the operation.
6. The Action Agent runs the Unity Catalog Function through its MCP server, which writes to Databricks (for example, inserting the support case), and returns the confirmation.
7. The Supervisor combines the facts and the action confirmation into one grounded answer.
8. The answer flows back through FastAPI to the React UI, where it is shown to the user.

5. Data Model

Three Delta tables hold the support data, in the schema workspace.support_agent. The Data Agent reads from these tables; the Action Agent also writes to them - correcting invoices, inserting new support cases, and updating account status.

5.1 customers

Customer accounts. Fields: customer_id, customer_name, plan, account_status, contact_email.

5.2 invoices

Billing records. Fields: invoice_id, customer_id, customer_name, month, amount_billed, amount_expected, status (Correct / Overbilled), issue_reason. The difference between amount_billed and amount_expected is the overbilling amount.

5.3 support_tickets

Past support cases. Fields: ticket_id, customer_id, customer_name, issue_type, description, resolution, status, created_date.

6. Setup & Running the Project

This section covers what a new person needs to get the project running. It assumes the Databricks workspace and the Google Gemini API key already exist; it does not cover creating those accounts.

6.1 Prerequisites

- Python 3.11
- Node.js v22 or later (for the React frontend)
- Git, and access to the project repository
- A .env file with valid credentials (Databricks + Gemini). The data, SQL warehouse, and Genie Space already exist in the shared Databricks workspace these credentials point to - no data setup is required.

6.2 Project Structure

The backend is organised as follows:

backend/		
agents/	supervisor.py	(the Supervisor / orchestrator)
	data_agent.py	(the Data Agent)
	action_agent.py	(the Action Agent)

```
mcp_servers/  sql_server.py      (SQL / Genie MCP server, port 8000)
              action_server.py  (UC Functions MCP server, port 8002)
api/          main.py           (FastAPI /ask endpoint -> Supervisor)
test_sql_mcp.py test_action_mcp.py (MCP tool tests)
requirements.txt
frontend/                                           (React + Vite chat UI)
src/App.jsx  src/App.css                          (the UI)
.env                                                (credentials - not committed)
```

6.3 Environment Variables

Create a .env file in the project root with the following keys:

```
DATABRICKS_HOST=<workspace-host, no https://>
DATABRICKS_TOKEN=<personal access token>
DATABRICKS_HTTP_PATH=<sql warehouse http path>
DATABRICKS_CATALOG=workspace
DATABRICKS_SCHEMA=support_agent
DATABRICKS_GENIE_SPACE_ID=<genie space id>
GOOGLE_API_KEY=<gemini api key>
```

The Databricks token must include the sql and genie scopes (and aisearch for the future Search Agent). The .env file is gitignored and must never be committed.

6.4 Installing Dependencies

Backend - from the backend folder, create a virtual environment and install the requirements:

```
cd backend
python -m venv venv311
venv311\Scripts\activate      (Windows)
pip install databricks-sdk databricks-sql-connector
pip install -r requirements.txt
```

Frontend - from the frontend folder, install the Node dependencies:

```
cd frontend
npm install
```

6.5 Running the Project

The system runs as four processes, each in its own terminal. The Supervisor calls both specialist agents, so both MCP servers must be running. Start them in this order.

Terminal 1 - SQL / Genie MCP server (backend, virtual environment active). Serves the Data Agent on port 8000:

```
cd backend
venv311\Scripts\activate
python mcp_servers/sql_server.py
```

Terminal 2 - UC Functions MCP server (backend, virtual environment active). Serves the Action Agent on port 8002:

```
cd backend
venv311\Scripts\activate
python mcp_servers/action_server.py
```

Terminal 3 - FastAPI backend (backend, virtual environment active). Calls the Supervisor:

```
cd backend
venv311\Scripts\activate
```

```
uvicorn api.main:app --reload --port 8001
```

Terminal 4 - React frontend (frontend folder, no virtual environment):

```
cd frontend  
npm run dev
```

Then open <http://localhost:5173> in a browser. Both MCP servers (ports 8000 and 8002) must be running before FastAPI (port 8001), since the agents connect to them over HTTP; the React app (port 5173) calls FastAPI, which calls the Supervisor. To verify the MCP tools independently, run `test_sql_mcp.py` and `test_action_mcp.py`.

7. Key Engineering Decisions

7.1 Custom MCP Server (not the managed one)

Databricks offers managed MCP servers, but we built our own with FastMCP. This gives full control over the tools exposed, keeps the architecture consistent across all agents, and demonstrates the MCP pattern directly. The native Databricks SQL connector is used inside our server as the implementation detail.

7.2 HTTP Transport (not stdio)

MCP servers can communicate over stdio (as a subprocess) or HTTP (as a standalone service). We use HTTP. The server runs as its own process, which keeps it independent and reusable by future agents, and avoids an environment-specific issue with the Databricks connector under stdio (see Section 9).

7.3 Fixed Tools and Genie Together

Rather than choosing between pre-written queries and natural-language querying, the server provides both. Fixed tools give speed and predictability for common questions; Genie gives flexibility for open-ended ones. The agent selects between them per question.

7.4 Model-Agnostic Framework

Building on LangChain keeps the agents independent of any single LLM. The current model (Gemini) can be swapped for another provider or an enterprise-hosted model by changing one configuration, with no change to the agents or tools.

7.5 Specialist Agents as Tools

The Supervisor treats each specialist agent as a single callable tool rather than connecting to MCP servers itself. This keeps orchestration cleanly separated from each specialist's internal logic, lets the specialists be developed and tested in isolation, and makes the system extensible - a new specialist is added as one more tool on the Supervisor without touching its routing code.

7.6 Python Unity Catalog Functions for Write Actions

SQL Unity Catalog Functions can only return values, not modify tables, so they cannot perform the Action Agent's write operations. The write actions are therefore implemented as Python Unity Catalog Functions, which authenticate explicitly and run the UPDATE or INSERT against the Delta tables. This keeps the actions real and verifiable while remaining governed Unity Catalog objects.

8. Security & Governance

- Parameterised queries - the fixed tools pass user values as query parameters, never concatenated into SQL, preventing injection.
- Scoped access tokens - the Databricks token is limited to the scopes the agents need (sql, genie), so it can only reach permitted capabilities.

- Unity Catalog - governs access to all data and tools in Databricks.
- Credential isolation - all secrets live in the gitignored .env file and are never committed to the repository.

Known limitation: the Python Unity Catalog Functions that perform write actions currently authenticate with a token embedded in the function body inside Databricks. This is acceptable for the MVP - the token is not in the code repository - but for production it should be replaced with a securely managed credential (for example a Databricks secret or service principal) rather than an embedded token.

9. Problems Encountered & Solutions

Recorded concisely so the next person can avoid the same issues.

9.1 MCP tool execution hung under stdio transport

Tool discovery worked but tool execution hung indefinitely with no error. Cause: the Databricks SQL connector does not complete its connection when run inside an MCP stdio subprocess on Windows. Solution: switch the MCP server to HTTP transport, so it runs as its own process where the connector works normally.

9.2 Blocking calls froze the async server

The Databricks connector is synchronous and blocked the server's async event loop. Solution: run the blocking database calls in a separate thread (`asyncio.to_thread`) inside the async tools.

9.3 Databricks token scope errors

Calls failed with "required scopes: sql" and later "required scopes: genie". Cause: the access token was missing the needed scopes. Solution: regenerate or update the token to include sql and genie (and aisearch for the future Search Agent).

9.4 Gemini API key format

Newer Google API keys (AQ-prefixed) initially failed with a 401, while older (Alza-prefixed) keys worked. This is a known library-version compatibility issue with the key format. Solution: use a compatible key, or ensure the Google client library is up to date.

9.5 SQL UC Functions could not perform write actions

The Action Agent's case-creation was first written as a SQL Unity Catalog Function, but SQL functions can only return a value - they cannot INSERT or UPDATE - so the action did not actually change any data. Solution: implement the write actions as Python Unity Catalog Functions, which can run UPDATE and INSERT statements against the tables.

9.6 Python UC Function default authentication failed

The first Python Unity Catalog Function failed at runtime with "cannot configure default credentials" - the client inside the function had no credentials to authenticate with. Solution: construct the Databricks client with explicit host and token rather than relying on default credential discovery. With explicit authentication the function executed and the writes landed.

9.7 Gemini free-tier rate limit

The free Gemini tier allows a limited number of requests per day per model. Because the Supervisor and both specialists each make their own model calls, a single combined question can use several requests, so the limit is reached quickly during heavy testing. Workaround: switch the model to a different Gemini model (separate quota) or a higher-limit provider; LangChain makes this a one-line change.

10. Technology Stack

Layer	Technology & Role
Orchestration	Supervisor agent (LangChain) - routes to specialists and combines results
Specialist Agents	Data Agent (retrieval) and Action Agent (operations), each a LangChain agent
Reasoning / Agent	LangChain reasoning loop - plans, selects tools, composes answers
LLM	Google Gemini - model-agnostic, swappable via LangChain
Integration	Custom MCP server (FastMCP) + langchain-mcp-adapters
Tools	Fixed parameterised SQL tools + Genie natural-language tool
Actions	Unity Catalog Functions - customer lookup plus invoice correction, case creation, and account status updates (Python UC Functions for writes)
Analytics	Databricks Genie - natural language to SQL, asynchronous
Data	Databricks Delta tables via serverless SQL warehouse
Governance	Unity Catalog + scoped access tokens
Frontend / API	React (Vite) chat UI + FastAPI /ask endpoint
Language / Runtime	Python 3.11 · Node.js v22
Version Control	GitHub

11. Future Enhancements

The Supervisor, Data Agent, and Action Agent are built and working together. The architecture is designed to extend further; the following are the natural next steps.

- Search Agent - a specialist that retrieves past tickets and knowledge-base articles by semantic similarity via a Databricks AI Search MCP server. It plugs into the existing design as one additional tool on the Supervisor, requiring no change to the orchestration logic. This extends the system from structured data and actions to unstructured knowledge retrieval.
- Production credential handling - replace the token embedded in the Python UC Function bodies with a Databricks secret or service principal.
- Conversational memory - multi-turn context held at the Supervisor level, so follow-up questions can build on earlier ones.
- Higher-throughput LLM - move from the free Gemini tier to a higher-limit or enterprise-hosted model for sustained use, a one-line change thanks to the model-agnostic framework.